26F Stat TA Session W11

1. Data Wrangling

- We have already learn many technics to deal with the raw data. This week we are going to go through more technics in data wrangling

Number of Rows/ Columns `nrow()` , `ncol()` , `dim()`

- This is the very first step of the data wrangling work! To see how many samples you have in the dataset.

```
df <- data.frame(
    name = c("Alice", "Bob"),
    score = c(80, 90),
    pass = c(TRUE, TRUE)
    )

nrow(df) # 2
ncol(df) # 3
dim(df) # 2 3
```

Trim String `substring()`

- Recall that in W6, we were introduced to the function `paste()` to paste characters together into a "superstring." `substring()` is similar to the inverse of `paste()` , which allows us to trim the original string into a substring

```
# syntax
substring(str, start_index, end_index)

# example
str1 <- "Cat_and_Dog"
substr1 <- substring(str1, 1, 3)
print(substr1)

# output
[1] "Cat"
```

Create Vector in Sequence `seq()`

- Sometimes you may want to create a sequence (e.g., arithmetic sequence)
    - When? Think about it

```r
# syntax
seq(from , to, by)

# example
seq(0, 100, 7)

# output
[1] 0 7 14 21 28 35 42 49 56 63 70 77 84 91 98
```

Date data

```r
# example
begin_date <- as.Date("2026-01-01")
end_date <- as.Date("2026-12-31")
print(seq(begin_date, end_date, "1 months"))

# output
[1] "2026-01-01" "2026-02-01" "2026-03-01" "2026-04-01" "2026-05-01" "2026-06
-01"
[7] "2026-07-01" "2026-08-01" "2026-09-01" "2026-10-01" "2026-11-01" "2026-12
-01"
```

`seq_along()`

- A similar function as `seq()` is `seq_along()` . It allows us to create a sequence at the length of a specific object

```r
x <- c("A", "B", "C", "D")
seq_along(x)

# output
[1] 1 2 3 4
```

- One common usage of `seq_along()` is in a `for` loop

```r
x <- c(10, 20, 30)
for (i in seq_along(x)){
    print(x[i])
}

# output
[1] 10
```

```
[1] 20
[1] 30
```

Remove Missing Values `na.omit()`

- In many cases, the dataset includes missing values (`NA`). How to deal with missing values depends largely on what you want to see from the data. A common (and naive) way is simply to delete them!
- `na.omit()` removes all `NA` in vectors or dataframes but keeps the attribute (extra information attached to an object)

```r
# example
x <- c(1, NA, 3, NA, 5)
x_clean <- na.omit(x)
print(x_clean)

# output
# keeps an attribute (so you can see the deleted values)
[1] 1 3 5
attr(,"na.action") # you can use as.vector() to remove these
[1] 2 4
attr(,"class")
[1] "omit"

# example
df <- data.frame(
        x = c(1, NA, 3, 4, NA),
        y = c(10, 20, NA, 40, 50)
        )

df_clean <- df %>%
  na.omit()
print(df_clean)

# output
  x  y     # sometimes na.omit() may be too aggressive...
1 1 10
4 4 40
```

A less Aggressive Alternative... `drop_na()`

- If we only want to deal with `NA` in a specific column (e.g. `x`), we can use `drop_na()` in the package `tidyr()` instead
  - `tidyr()` is also a member in the family of `tidyverse()` (just like `ggplot2` and `dplyr`)

```r
df %>%
    drop_na(x)  # delete when x is NA

# output
    x  y
1 1 10
3 3 NA
4 4 40

df %>%
    drop(x, y)  # delete when x or y is NA

# output
  x  y
1 1 10
4 4 40
```

Calculations with NA Values `na.rm`

- When calculating the statistics (e.g., the mean), a single `NA` value can invalidate the whole calculation. Helpfully, the argument `na.rm` spares us the time for going through the above steps of cleaning `NA` values
    - The `na.rm` argument does not change the original data. It's only an argument for the calculation

```r
score <- c(1, 2, NA, 4)
mean(score)

# output
[1] NA

mean(score, na.rm = TRUE)

# output
[1] 2.333333   # = (1+2+4)/3
```

Find patterns inside element `grepl()`

- `grepl()` check if a pattern (regular expression) is observed inside an element
- It returns a logical vector: `TRUE` if the pattern is inside that element. `FALSE` otherwise

```r
# syntax
grepl(pattern, str)
grepl(pattern, str_vec)
```

```r
# example
grepl("app", "apple") # TRUE

fruits <- c("apple", "banana", "pineapple", "pear")
grepl("apple", fruits) # TRUE FALSE TRUE FALSE

strings <- c("abcba", "acbcb", "abbbc", "babcc")
grepl("abc", strings) # TRUE FALSE FALSE TRUE
```

Exercises

W11 Exercises